

Chico Ridership forecast

ROWAN MORKNER

2025-12-18

```
library(readxl)
library(dplyr)
library(tidyr)

load_xlsx_sheets <- function(file_path) {
  # Get all sheets names
  sheet_names <- excel_sheets(file_path)

  # Read each sheet and assign to a variable with the sheet name
  for (sheet in sheet_names) {
    df <- read_excel(file_path, sheet = sheet)
    assign(sheet, df, envir = .GlobalEnv)
  }

  message("Loaded ", length(sheet_names), " sheets into data frames: ",
    paste(sheet_names, collapse = ", "))
}

load_xlsx_sheets("transit_data.xlsx")

rm(`Mozart Reports`, `Read Me`, `UPT Estimates`, `VRM Estimates`, `Calendar Year UPT`, `Calendar Year V`)

library(dplyr)
library(stringr)

# These match the "Primary UZA Name" column in the NTD yearly data
major_city_uzas <- c(
  "Chico, CA"
)

# Create a filtering function to reuse
filter_major_cities <- function(df) {
  df %>%
    filter(`UZA Name` %in% major_city_uzas)
}

UPT <- filter_major_cities(UPT)

rm(VRM, VOMS, VRH)

# create pivot longer function for transit data frames make the "value" column the name of the data frame
```

```

pivot_longer_transit <- function(df, value) {
  df_long <- df %>%
    pivot_longer(
      cols = matches("[0-9]{1,2}/[0-9]{4}$"), # Matches columns like "1/2002", "12/2025"
      names_to = "Month_Year",
      values_to = value
    ) %>%
    mutate(
      Month_Year = as.Date(paste0("01/", Month_Year), format = "%d/%m/%Y")
    )
  return(df_long)
}

UPT_long <- pivot_longer_transit(UPT, "UPT")

rm(UPT)

```

filter for important variables

```

UPT <- UPT_long %>%
  select(`UZA Name`, Month_Year, UPT, Mode, `3 Mode`)

# remove long data frames
rm(UPT_long)

```

create a function to sum data frames by UZA name, month year, mode

```

library(lubridate)

##
## Attaching package: 'lubridate'
## The following objects are masked from 'package:base':
##
##   date, intersect, setdiff, union
# split year and month into seperate columns

sum_funk <- function(df, value) {
  df_summed <- df %>%
    group_by(`UZA Name`, Month_Year, `Mode`) %>%
    summarise(
      !!sym(value) := sum(!!sym(value), na.rm = TRUE),
      .groups = "drop"
    )
  return(df_summed)
}

UPT_summed <- sum_funk(UPT, "UPT")

```

```

UPT <- UPT_summed %>%
  filter(Mode != "DR")

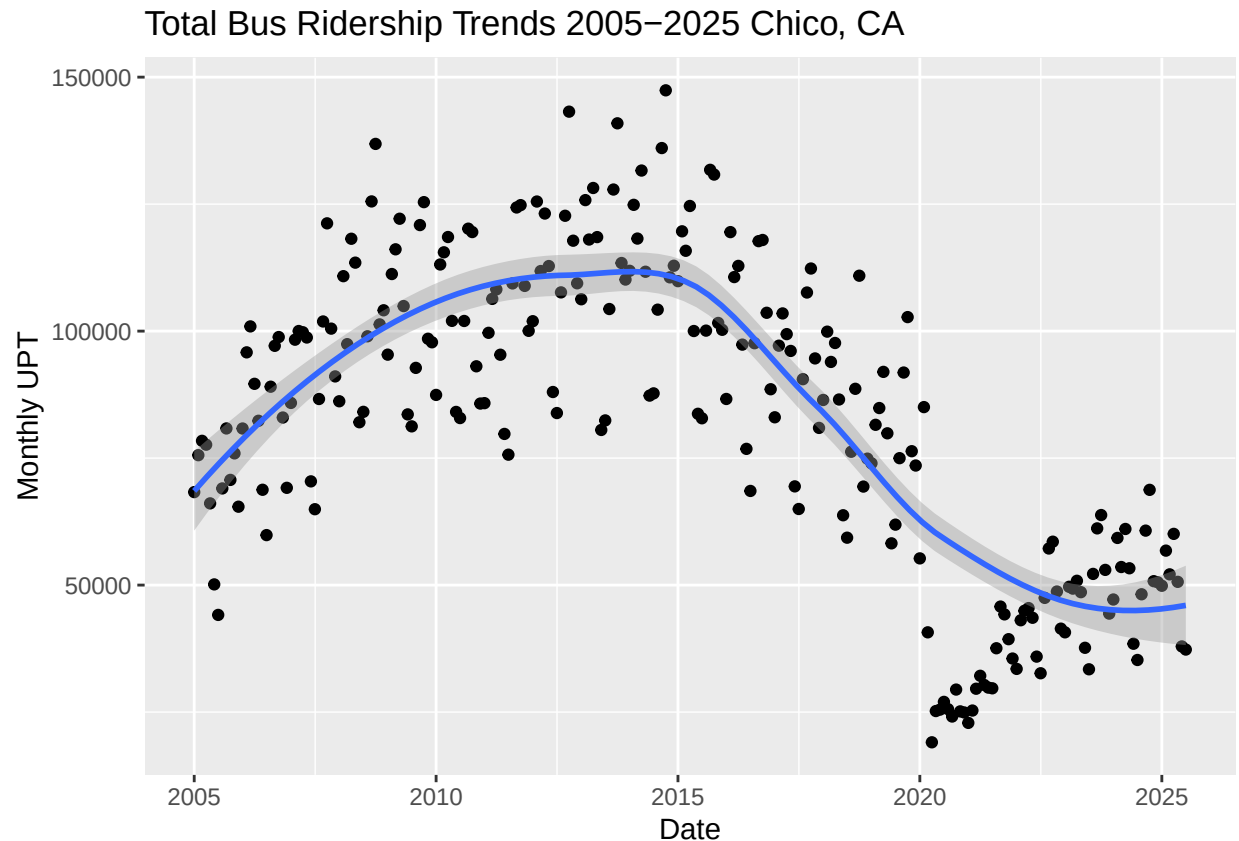
# trim down to years 2005-2024 & remove ferry mode
UPT_trimmed <- UPT %>%
  filter(Month_Year >= as.Date("2005-01-01") & Month_Year <= as.Date("2025-12-31")) %>%
  mutate(
    year = year(Month_Year),
    month = month(Month_Year)
  )

# sum upt by year
UPT_yearly <- UPT_trimmed %>%
  group_by(`UZA Name`, year) %>%
  summarise(
    UPT = sum(UPT, na.rm = TRUE),
    .groups = "drop"
  )

library(ggplot2)
ggplot(UPT_trimmed, aes(x = Month_Year, y = UPT))+
  geom_point()+
  geom_smooth(method= "loess")+
  labs(
    title = "Total Bus Ridership Trends 2005-2025 Chico, CA",
    x = "Date",
    y = "Monthly UPT"
  )

## `geom_smooth()` using formula = 'y ~ x'

```



Total ridership trends were already falling pre pandemic due to lower enrollment at CSU Chico, along with national trends such as the rise of ride-share platforms like Uber and Lyft.

During the pandemic, ridership fell off sharply, but has recovered off its lows. Ridership appears to be heading towards off around its pre-pandemic numbers around 750,000 UPT per month.

Ridership in exhibits strong seasonality patterns due to the influx of students during fall/ spring semesters, and the opposite during winter and summer breaks.

```
library(prophet)
```

```
## Loading required package: Rcpp
```

```
## Loading required package: rlang
```

```
# 1. Prepare data (prophet requires columns 'ds' and 'y')
```

```
df_prophet <- UPT_trimmed %>%  
  filter(Month_Year >= as.Date("2020-03-01") & Month_Year <= as.Date("2025-12-31")) %>%  
  rename(ds = Month_Year, y = UPT) %>%  
  mutate(cap = 75000)
```

```
# 2. Fit model with logistic growth
```

```
m <- prophet(df_prophet, growth = 'logistic', yearly.seasonality = TRUE)
```

```
## Disabling weekly seasonality. Run prophet with weekly.seasonality=TRUE to override this.
```

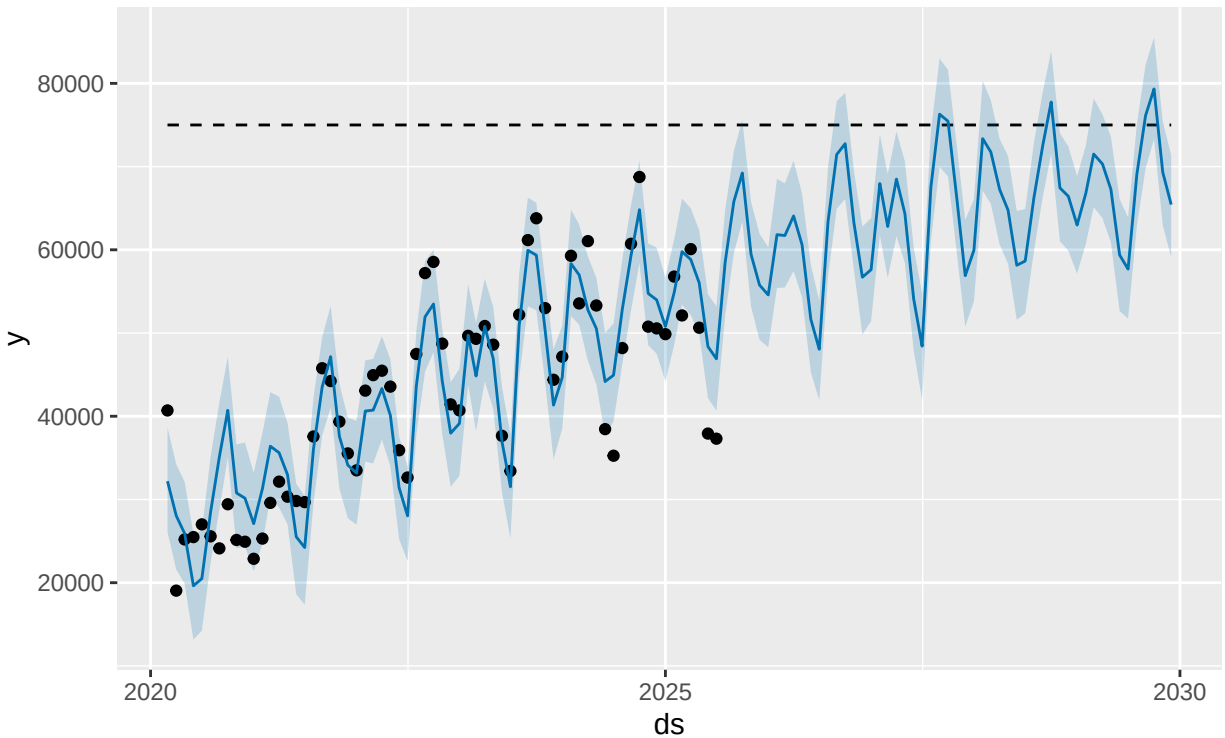
```
## Disabling daily seasonality. Run prophet with daily.seasonality=TRUE to override this.
```

```
# 3. Forecast 3 years (36 months)
```

```
future <- make_future_dataframe(m, periods = 53, freq = 'month') # The number of months from the end of
```

```
future$cap <- 75000 # Keep the cap consistent
forecast <- predict(m, future)

plot(m, forecast)
```



Prophet is a procedure for forecasting time series data based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects. It works best with time series that have strong seasonal effects and several seasons of historical data. Prophet is robust to missing data and shifts in the trend, and typically handles outliers well.

Prophet is open source software released by Facebook's *Core Data Science team*. It is available for download on CRAN and PyPI.

Once we have montly forecast data, we can sum to yearly to extract an overall trend and project over the next 15 years assuming a moderate growth rate of 0.5%

```
library(dplyr)
library(lubridate)

# 1. Aggregate the stable Prophet forecast years
annual_forecast <- forecast %>%
  mutate(year = year(ds)) %>%
  group_by(year) %>%
  summarize(annual_upt = sum(yhat)) %>%
  filter(year >= 2020)

# 2. Define the steady-state growth for the remainder
```

```

# Assume 0.05% annual growth after the Prophet forecast ends
growth_rate <- 0.005
last_forecasted_year <- max(annual_forecast$year)
last_upt_value <- tail(annual_forecast$annual_upt, 1)

# 3. Project out to the 20th year
remaining_years <- (last_forecasted_year + 1):2045
steady_state_df <- data.frame(
  year = remaining_years,
  annual_upt = last_upt_value * (1 + growth_rate)^(seq_along(remaining_years))
)

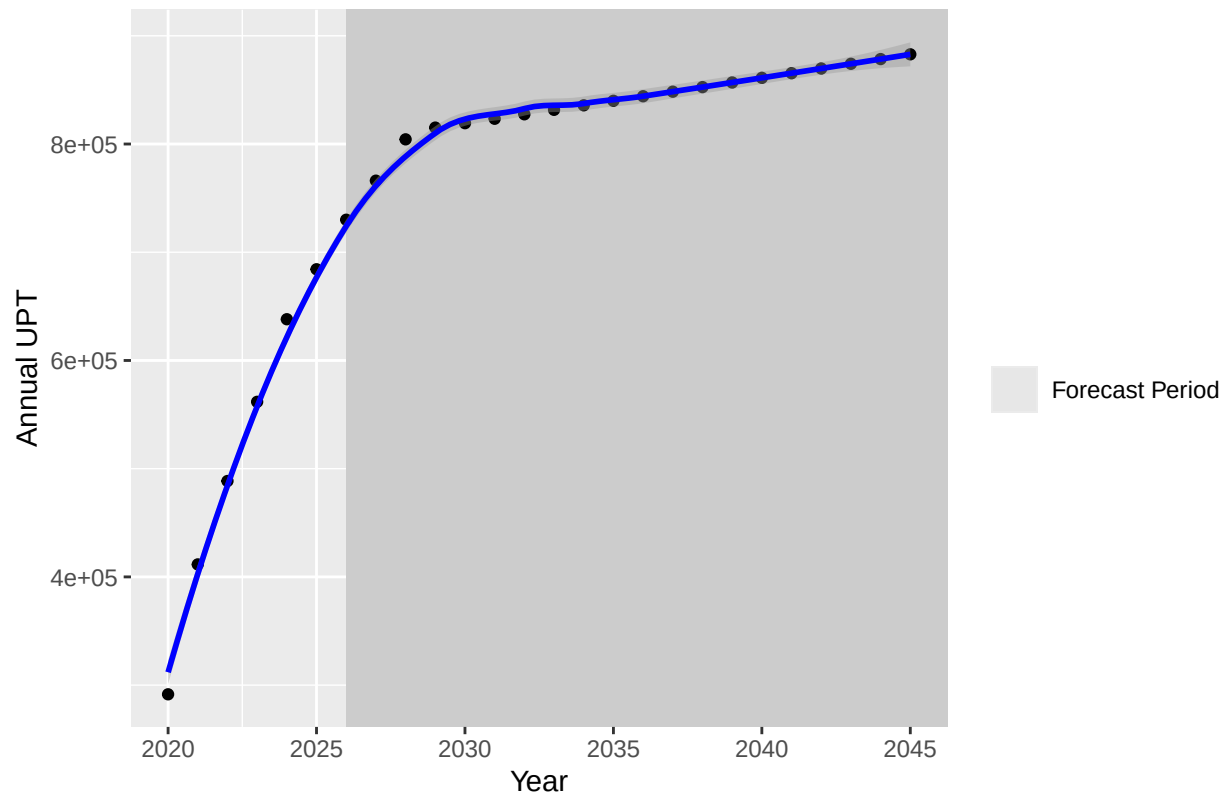
# 4. Combine into a final 20-year Baseline UPT table
bca_baseline_stream <- bind_rows(annual_forecast, steady_state_df)

ggplot(bca_baseline_stream, aes(year, annual_upt)) +
  geom_rect(aes(xmin = 2026, xmax = Inf, ymin = -Inf, ymax = Inf, fill = "Forecast Period"),
    alpha = 0.1) +
  geom_point() +
  geom_smooth(color = "blue") +
  scale_fill_manual(name = "", values = c("Forecast Period" = "grey80")) +
  labs(
    title = "UPT Forecast",
    x = "Year",
    y = "Annual UPT"
  )
)

## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'

```

UPT Forecast



```
library(dplyr)
```

```
library(dplyr)
```

```
# --- 1. SET ANCHOR SHARES & PROJECT PARAMETERS ---
```

```
# Based on the B-Line Ridership Analysis (2022 Factored Estimates)
```

```
system_total_2022 <- 532487
```

```
route_3_baseline_2022 <- 58400
```

```
route_14_baseline_2022 <- 29600
```

```
# Individual route shares of the total system
```

```
share_3 <- route_3_baseline_2022 / system_total_2022
```

```
share_14 <- route_14_baseline_2022 / system_total_2022
```

```
share_combined <- share_3 + share_14
```

```
# Service Day Parameters (Excluding Sundays as per B-Line schedule)
```

```
weekdays_per_year <- 255
```

```
saturdays_per_year <- 52
```

```
total_service_days <- weekdays_per_year + saturdays_per_year
```

```
# Build Scenario Parameters
```

```
start_year <- 2025
```

```
target_induced <- 65000
```

```
# --- 2. GENERATE 20-YEAR SEGMENTED RIDERSHIP BY ROUTE ---
```

```
bca_final_ridership <- bca_baseline_stream %>%
```

```

mutate(
  # Step A: Baseline Scaling for individual routes
  baseline_r3_annual = annual_upt * share_3,
  baseline_r14_annual = annual_upt * share_14,
  baseline_combined_corridors = annual_upt * share_combined,

  # Step B: Segment Route 3 into Weekday vs Saturday
  baseline_r3_weekday = baseline_r3_annual * (weekdays_per_year / total_service_days),
  baseline_r3_saturday = baseline_r3_annual * (saturdays_per_year / total_service_days),

  # Step C: Segment Route 14 into Weekday vs Saturday
  baseline_r14_weekday = baseline_r14_annual * (weekdays_per_year / total_service_days),
  baseline_r14_saturday = baseline_r14_annual * (saturdays_per_year / total_service_days),

  # Step D: Calculate 3-Year Linear Ramp-Up Factor
  ramp_up_factor = case_when(
    year == 2025 ~ 1/3,
    year == 2026 ~ 2/3,
    year >= 2027 ~ 1.0,
    TRUE ~ 0
  ),

  # Step E: Apply Induced Boardings to Weekdays (Proportional to Route Share)
  # Total induced demand is allocated based on the relative size of each route
  induced_r3_weekday = (target_induced * (share_3 / share_combined)) * ((1 + growth_rate)^(year - start_year)),
  induced_r14_weekday = (target_induced * (share_14 / share_combined)) * ((1 + growth_rate)^(year - start_year)),

  # Step F: Final Build Scenario per Route (Weekday focus)
  build_r3_weekday = baseline_r3_weekday + induced_r3_weekday,
  build_r14_weekday = baseline_r14_weekday + induced_r14_weekday,

  # Final Annual Totals for each route
  build_r3_annual = build_r3_weekday + baseline_r3_saturday,
  build_r14_annual = build_r14_weekday + baseline_r14_saturday,

  # Combined Totals for the Corridor
  build_scenario_upt_annual = build_r3_annual + build_r14_annual
)

# --- 3. VIEW RESULTS ---

write.csv(bca_final_ridership, "BCA_ridership.csv")
# remove all

df <- read.csv("BCA_ridership.csv")

# Create simplified tidy dataframe
library(tidyr)

bca_tidy <- df %>%
  select(year,
    baseline_r3_annual,
    baseline_r14_annual,

```



```

        induced_r3_weekday,
        induced_r14_weekday) %>%
pivot_longer(
  cols = c(baseline_r3_annual, baseline_r14_annual),
  names_to = "route",
  names_pattern = "baseline_r(\\d+)_annual",
  values_to = "baseline_upt"
) %>%
mutate(
  route = paste0("Route ", route),
  induced_upt = ifelse(route == "Route 3", induced_r3_weekday, induced_r14_weekday)
) %>%
select(year, route, baseline_upt, induced_upt)

ggplot(bca_tidy, aes(x = year)) +
  # 1. Baseline points and smooth line
  geom_point(aes(y = baseline_upt, color = "Baseline Forecast")) +
  geom_smooth(aes(y = baseline_upt, color = "Baseline Forecast")) +

  # 2. Induced + Baseline points and line
  geom_point(aes(y = induced_upt + baseline_upt, color = "Additional Riders")) +
  geom_line(aes(y = induced_upt + baseline_upt, color = "Additional Riders")) +

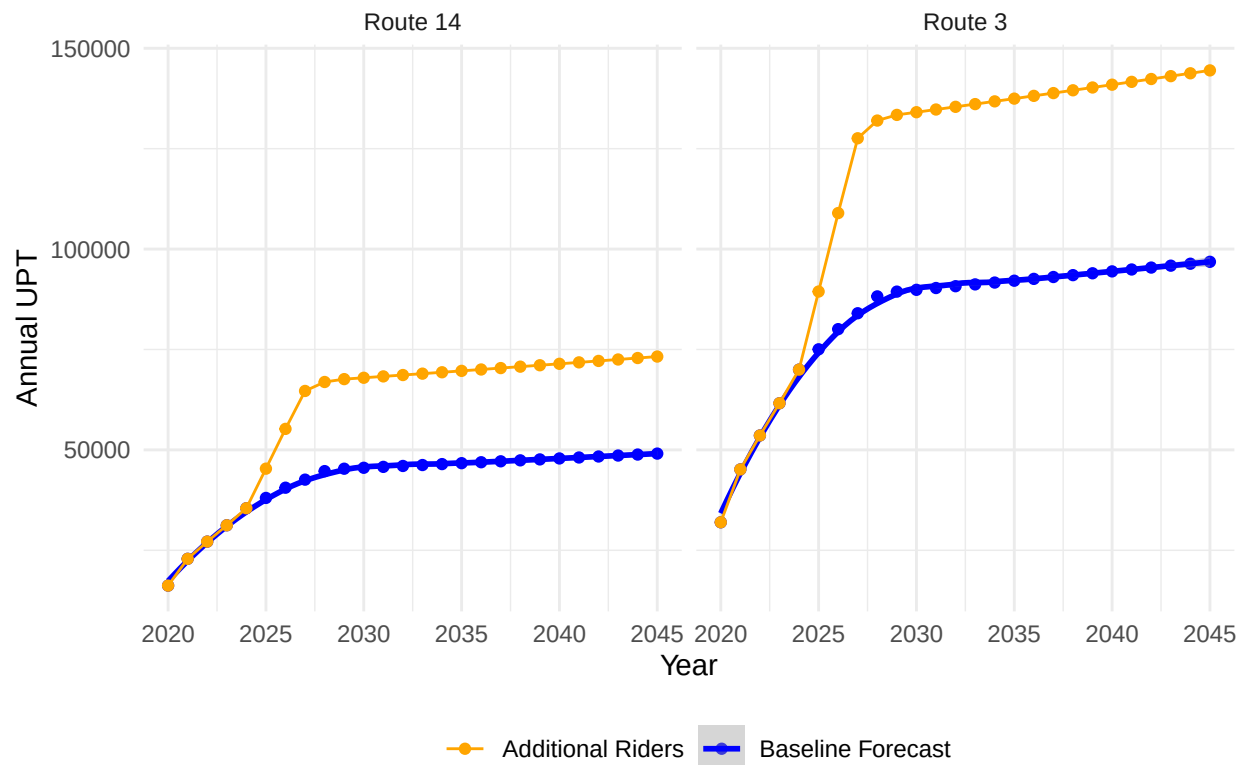
  # 3. Faceting and Titles
  facet_grid(~route) +
  labs(
    title = "BCA Ridership Projections by Route",
    x = "Year",
    y = "Annual UPT",
    color = ""
  ) +

  # 4. Define the manual colors and move legend to bottom
  scale_color_manual(values = c(
    "Baseline Forecast" = "blue",
    "Additional Riders" = "orange"
  )) +
  theme_minimal() +
  theme(legend.position = "bottom")

## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'

```

BCA Ridership Projections by Route



```
bca_wide <- df %>%
  select(year,
         baseline_r3_annual,
         baseline_r14_annual,
         induced_r3_weekday,
         induced_r14_weekday)

write.csv(bca_wide, "projected_UPT.csv")
```